

1. Calculate the total transaction amount per customer, returning a Map<Long, Double>.

Explanation: You use the **Collectors.groupingBy()** collector to group the transactions by the customerId. The downstream collector is **Collectors.summingDouble()**, which efficiently calculates the sum of the amount field within each customer's group.

```
// Assuming a Transaction class exists with getCustomerId() and
getAmount()
List<Transaction> transactions = getTransactions();

Map<Long, Double> totalAmountPerCustomer = transactions.stream()
    .collect(Collectors.groupingBy(
        Transaction::getCustomerId, // Key: Group by customer ID
        Collectors.summingDouble(Transaction::getAmount) // Value: Sum
        the double amount field
    ));
```

2. Find the top 3 most frequent words and return them with their counts.

Explanation: First, use **flatMap** to split the paragraph into words and then **groupingBy(counting)** to get the frequency map. Then, stream over the map's entries, sort them by count in reverse order, and use **limit(3)** to take the top three.

```
String paragraph = "The quick brown fox jumps over the lazy dog. The dog
is quick.";
Map<String, Long> topWords =
Arrays.stream(paragraph.toLowerCase().split("\\W+")) // 1. Split into
words
    .filter(s -> !s.isEmpty())
    .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting())) // 2. Count frequencies
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed()) // 3.
Sort by count (descending)
    .limit(3) // 4. Take the top 3
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
(e1, e2) -> e1, LinkedHashMap::new)); // Collect into a Map preserving
order
// Result: {the=3, quick=2, dog=2}
```

3. Compute the average salary per department, returning a Map<Department, Double>.

Explanation: You use the **Collectors.groupingBy()** collector to group employees by their Department. The downstream collector is **Collectors.averagingDouble()**, which efficiently computes the average of the salary field for all employees within that department group.

```
// Assuming Employee has getDepartment() and getSalary()
List<Employee> employees = getEmployeeList();

Map<Department, Double> averageSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment, // Key: Group by department object
        Collectors.averagingDouble(Employee::getSalary) // Value:
        Calculate the average salary
    ));
```

4. Generate an infinite stream of integers starting from 1, filter even numbers, and then take only the first 5 even numbers as a List<Integer>.

Explanation: You use **Stream.iterate()** to generate the infinite stream (starting at 1 and incrementing by 1). You use **filter()** to keep only even numbers, and then use the **limit(5)** intermediate operation to stop the infinite stream after 5 elements have been found.

```
List<Integer> firstFiveEvens = Stream.iterate(1, n -> n + 1) // Start at
1, increment by 1 (infinite)
    .filter(n -> n % 2 == 0) // Filter: Keep only even numbers
    .limit(5) // Stop the stream after 5 elements are found
    .collect(Collectors.toList());
// Result: [2, 4, 6, 8, 10]
```

5. Find the first non-empty Optional<String> and return its value as Optional<String>.

Explanation: You use the **filter()** operation to keep only Optional objects that are present (isPresent()). Then, you use **findFirst()** to get the first one. Finally, you use **map(Optional::get)** to transform the Optional<Optional<String>> into an Optional<String> containing the value.

```
Stream<Optional<String>> optionals = Stream.of(Optional.empty(),
Optional.of("A"), Optional.of("B"));

Optional<String> firstNonEmpty = optionals
    .filter(Optional::isPresent) // Filter out Optional.empty()
    .map(Optional::get) // Extract the String value from the Optional
    .findFirst();
// Result: Optional["A"]
```

6. Produce a `Map<Department, List<Employee>>` mapping each department to its employees.

Explanation: This is a direct use case for the `Collectors.groupingBy()` collector. The grouping key extracts the Department from the Employee, and the default collector (`Collectors.toList()`) groups all employees sharing that key into a single list.

```
// Assuming Employee has a Department object
List<Employee> employees = getEmployeeList();

Map<Department, List<Employee>> employeesByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment)); // Key:
    Department, Value: List of Employees
```

7. Group words into anagram clusters but only return groups that have at least 2 words.

Explanation: You group words by their canonical (sorted) character string. Then, you stream over the map's entries and use `filter()` to keep only groups where the size of the value list is ≥ 2 . You collect the results into a new map preserving the grouping.

```
List<String> words = Arrays.asList("listen", "silent", "car", "arc",
    "tool");
Map<String, List<String>> anagramClusters = words.stream()
    .collect(Collectors.groupingBy(word -> {
        char[] chars = word.toCharArray();
        Arrays.sort(chars);
        return new String(chars); // Create the canonical key (e.g.,
        "eilnst")
    })))
    .entrySet().stream()
    .filter(entry -> entry.getValue().size() >= 2) // Filter: Keep only
    groups of size 2 or more
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
// Result: {"eilnst": [listen, silent], "acr": [car, arc]}
```

8. Compute the median salary, and what considerations would you mention about performance and memory when the list becomes very large?

Explanation: You find the median by sorting the salaries and then finding the middle element's index.

```
List<Double> salaries = getSalaryList();
List<Double> sortedSalaries =
    salaries.stream().sorted().collect(Collectors.toList());
int size = sortedSalaries.size();
```

```
double median = (size % 2 != 0)
    ? sortedSalaries.get(size / 2)
    : (sortedSalaries.get(size / 2 - 1) + sortedSalaries.get(size / 2))
    / 2.0;
```

Considerations: When the list becomes very large (e.g., billions of records), this approach fails because:

1. **Memory:** Loading all salaries into a List causes an **OutOfMemoryError** (OOM) because the heap cannot hold the data.
2. **Performance:** The **sorted()** operation requires $O(N \log N)$ time, which becomes extremely slow for massive datasets. For truly large data, sorting should be delegated to the database (SQL ORDER BY) or a stream processing engine.

9. Return a list of prime numbers up to 1 million using Streams (possibly **parallelStream**).

Explanation: You use **IntStream.rangeClosed** to generate numbers. You use **parallel()** to process chunks of numbers concurrently on multiple cores. You apply a helper filter function (**isPrime**) to test primality and collect the results.

```
// Assume MathUtils.isPrime(int n) exists
List<Integer> primeNumbers = IntStream.rangeClosed(2, 1_000_000)
    .parallel() // Process numbers on multiple threads for speed
    .filter(MathUtils::isPrime) // Filter using the primality check
    .boxed() // Convert primitive int back to Integer objects
    .collect(Collectors.toList());
```

Performance and Parallelism Trade-offs:

- **Parallel Benefit:** The primality check (**isPrime**) is a CPU-intensive task, making it ideal for **parallel streams**. Processing the numbers concurrently across multiple cores provides significant speedup.
- **Parallel Overhead:** For simple tasks (like just filtering numbers < 1000), the overhead of managing threads and combining results might outweigh the benefit, making a sequential stream faster. Here, the expensive CPU work justifies parallelism.